

Домашнее задание - 1

Падалко Александр, Б05-615

1 октября 2026 г.

Задача 1. $T(n) = 3T(\lfloor \sqrt{n} \rfloor) + \log_2 n$

Решение. Пусть $n = 2^m$, $m = \log_2 n$, $\sqrt{n} = \frac{m}{2}$

$$T(n) = S(m) = 3S(\frac{m}{2}) + m$$

$S(m)$ можем найти с помощью мастер-теоремы о рекурсии, в частности $S(m) = a \cdot S(\frac{m}{b}) + m^c$, где $a = 3, b = 2, c = 1$.

Отсюда следует, что поскольку $\log_a b = \log_2 3 > 1 = c$, то $S(m) = \Theta(m^{\log_2 3})$

Значит $T(n) = S(m) = \Theta(m^{\log_2 3}) = \Theta((\log_2 n)^{\log_2 3})$

□

Задача 2. $T(n) = 2T(\lceil \frac{n}{2} \rceil) + n \log_2 n$

Решение. Рассмотрим, как выглядит $T(n)$ при разделении на уровни.

В частности, на первом (самом высоком) уровне имеет просто $n \log_2 n$. Затем $2 \cdot \frac{n}{2} (\log_2 \frac{n}{2}) = n(\log_2 n - 1)$. На уровне k имеем: $n(\log_2 n - k)$. Значит, по сути, хотим найти $\sum_0^{\log_2 n} n(\log_2 n - k) = n \cdot \sum_0^{\log_2 n} (\log_2 n - k) = n \cdot \log_2 n (\log_2 n + 1) = \Theta(n \log_2^2 n)$

□

Задача 3. $T(n) = \sqrt{n} \cdot T(\lfloor \sqrt{n} \rfloor) + n$

Решение. Пусть $S(n) = \frac{T(n)}{n}$

$$S(n) = \frac{T(n)}{n} = \frac{T(\lfloor \sqrt{n} \rfloor)}{\sqrt{n}} + 1 = S(\sqrt{n}) + 1$$

Пусть $n = 2^m$, $m = \log_2 n$, $\sqrt{n} = \frac{m}{2}$

$$S(n) = P(m) = P(\frac{m}{2}) + 1 = \Theta(\log_2 m) = \Theta(\log_2 \log_2 n)$$

$$T(n) = S(n) \cdot n = \Theta(n \cdot \log_2 \log_2 n)$$

□

Задача 4. $T(n) = T(f(n)) + T(n - f(n)) + 1 + \log_2 \min(f(n), n - f(n))$, $f(n)$ — целое число от 1 до $n - 1$.

Решение. Рассмотрим случай $n - f(n) = f(n)$, т.е. $f(n) = \frac{n}{2}$.

$$\text{Тогда } T(n) = 2 \cdot T(\frac{n}{2}) + 1 + \log_2 \frac{n}{2} = 2 \cdot T(\frac{n}{2}) + \log_2 n$$

Пусть $n = 2^m$, $m = \log_2 n$

$$T(n) = S(m) = 2 \cdot S(m - 1) + m$$

Разобьём S на P и L , где $P(m) = 2 \cdot P(m - 1)$, а $L(m) = 2 \cdot L(m - 1) + m$ (то есть первое это просто степенная, а вторая, то сколько накопится лишнего относительно степенной).

Тогда очевидно, что $P(m) = \Theta(2^m)$.

Поскольку остатков накапливается по m с каждого значения, попробуем представить $L(m)$, как некоторый многочлен: $L(m) = a_0 \cdot m^0 + a_1 \cdot m^1 + \dots + a_k \cdot m^k (a_k \neq 0)$

$L(m) = 2 \cdot L(m-1) + m$. Пусть старшая степень больше 1.

Коэффициенты при старшей степени с левой и правой сторон должны быть равны. Причем с левой стороны коэффициент равен a_k , с правой стороны $2a_k$, следовательно $a_k = 0$ (противоречие).

Значит $L(m) = a_0 + a_1 \cdot m = 2 \cdot L(m-1) + m = 2 \cdot (a_0 + a_1 \cdot (m-1)) + m = 2(a_0 - a_1) + (2a_1 + 1) \cdot m \implies 2a_1 + 1 = a_1 \implies a_1 = -1 \implies 2a_0 + 2 = a_0 \implies a_0 = -2$. Значит $L(m) = -m - 2$.

Значит $S(m) = P(m) + L(m) = \Theta(2^m) - m - 2 = \Theta(2^m) = \Theta(n) = T(n)$.

Докажем, что $T(n) = \Theta(n)$. Для этого нам нужно доказать, что существуют c_1 и c_2 такие, что $c_1 n \leq T(n) \leq c_2 n$.

$T(n) \geq c_1 n$, так как по итогу по сути, дерево разбиения представляет собой некоторое бинарное дерево с n листьями. Тогда так как при каждом разбиении добавляется 1, то сумма по всем $(n-1) = \Theta(n)$.

Теперь докажем, что $T(n) \leq An - B \log_2 n$. Рассмотрим по индукции. $T(1)$ - константа, значит найдем такие A и B (при необходимости потом учтем, чтобы для $T(1)$ ничего не сломалось).

Пусть m - минимальное из n и $(n - f(n))$. Тогда $T(n) = T(m) + T(n-m) + 1 + \log_2 m \leq Am - B \log_2 m + A(n-m) - B \log_2 (n-m) + 1 + \log_2 m = An - B \log_2 m - B \log_2 (n-m) + 1 + \log_2 m = An - (B-1) \log_2 m - B \log_2 (n-m) + 1$.

$n-m > \frac{n}{2} \implies \log_2(n-m) \geq \log_2 n - 1 \implies$

$T(n) \leq An - (B-1) \log_2 m - B(\log_2 n - 1) + 1 \leq An - B \log_2 n$.

Получаем, что $T(n) = \Theta(n)$.

□

Задача 5. Реализовать класс *deque* с операциями: *push_back*, *pop_back*, *push_front*, *pop_front*, *operator[]*.

Учетное время всех операций должно быть $\mathcal{O}(1)$.

Решение. Рассмотрим такую структуру:

Будем хранить некоторый буфер, условно разделенный на 2 части. А именно - часть для *push* и часть для *pop*. По сути, воспринимаем их как 2 стека, к которым можно обращаться не только к самому верхнему элементу, а любому.

Когда приходят операции *push_back* и *pop_back* мы проделываем их со второй частью, если она не пуста, иначе с первой. Аналогично операции *push_front* и *pop_front* мы проделываем с первой частью, если она не пуста, иначе со второй. На самом деле, деление условное, так как, по сути, мы просто храним два индекса в буфере, которые считаем началом и концом нашего дека соответственно.

Если один из концов достигает границы допустимого буфера и требует еще расширения (то есть выйти за пределы буфера), то мы выделяем новый буфер размера = $4 \cdot$ количество элементов в деке (если количество нечетное, то увеличиваем до ближайшего четного).

Далее выделяем первую половину имеющихся элементов и помещаем их в конец области 2 (на рисунке все области одинакового размера - области нового выделенного буфера), то есть последний элемент первой области должен быть на последнем индексе в области 2.

Далее оставшиеся элементы перекидываем в начало области 3. Таким образом получим, что элементы снова представляют собой последовательный блок, но теперь расположенный в другой области памяти. Новая область памяти в 2 раза больше старой:

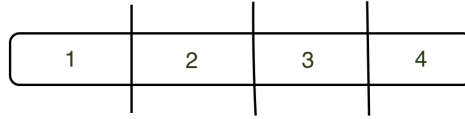


Рис. 1: Части нового буфера

Теперь проанализируем полученную структуру.

1) Метод бухгалтерского учета:

Итак, пусть каждый элемент при добавлении будет иметь 6 монетки. Тогда при обычном *push* (без расширения) и *pop* будет тратиться одна его монетка. То есть можем отложить еще как минимум 4 монеты "в копилку" с каждого элемента.

Теперь рассмотрим *push* с расширением. Предполагаем, что выделение свободного буфера не тратит времени. Итак, с предыдущего расширения буфера произошло как минимум размер одной четверти буфера операций *push*, иначе бы ни один из указателей не дошел бы до границы буфера. Имеем, что в таком случае в копилке накоплено как минимум 4 размера одной четверти буфера, то есть размер всего буфера. Тогда теперь можем потратить все монетки из "копилки" на перенос всего дека в новый буфер (по 1 монетке на каждый элемент). Получается, что мы никогда не залазим в долги по монетам, то есть после любых k операций $\sum_{i=1}^k c_i \leq \sum_{i=1}^k c'_i$, где c_i - реальная стоимость операции, а c'_i - потраченные монетки.

Значит после всех n операций мы потратили монеток не более $6 \cdot n = \Theta(n)$. Таким образом учетная стоимость любой операции $\mathcal{O}(1)$.

□

Задача 6. Дано 5 камешков и весы. Нужно отсортировать одними сравнениями двух камней и сделать это за минимальное количество взвешиваний.

Решение. Поскольку у нас есть $5! = 120$ исходов и каждое взвешивание делит это количество пополам (по сути, идем к листам бинарного дерева), то потребуется как минимум $\lceil \log_2 5! \rceil = \lceil \log_2 120 \rceil = 7$.

Докажем, что можем сделать это ровно за 7 взвешиваний.

Обозначим камешки как a, b, c, d, e .

Взвешивания по порядку:

1) a, b (Б.О.О. пусть $a < b$)

2) c, d (Б.О.О. пусть $c < d$)

3) b, d (Б.О.О. пусть $b < d$)

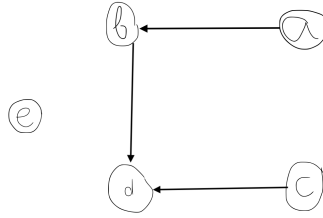


Рис. 2: После 3-х взвешиваний, стрелка указывает на больший из двух камешков (известные на данный момент связи)

4) b, e

1-й случай. Пусть $b < e$, тогда имеем такую картинку:

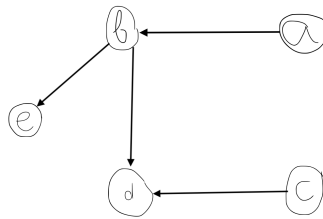


Рис. 3: В таком случае нам нужно сравнить d и e

Допустим $d < e$, тогда e наибольший. И у нас еще 2 взвешивания, чтобы определить место c (либо больше b , либо от a до b , либо до a ; 3 возможных позиции, значит за 2 взвешивания гарантированно сможем).

Пусть теперь $d > e$. Теперь сравним c и b , и затем сравним c либо с a , либо с e и тем самым определим итоговый порядок.

2-й случай $e < b$. Сравним теперь e и a и тем самым для тройки a, b, e поймем их окончательный порядок. Затем сравним второе число из них с c (это будет большее из a и e), и затем в зависимости от решения сравним c либо с меньшим из a и e , либо с b .

Таким образом, за 7 взвешиваний мы гарантированно найдем порядок камешков.

□

Задача 7. Есть n камешков, хотим найти второй минимум. Обозначим количество действий, за которое можем сделать это за $h(n)$.

Хотим оценить $2^{h(n)}$ асимптотически точно.

Решение. Для начала заметим, что $h(n) \leq n + \lceil \log_2 n \rceil - 2$.

Чтобы найти за $(n + \lceil \log_2 n \rceil - 2)$. По сути построим дерево отрезков на минимум на данных нам камешках за $(n - 1)$ взвешивание. То есть в листах у нас лежат сами камешки и мы постепенно идем вверх, каждый раз выбирая меньший (более легкий) из камешков. Тогда в корне дерева будет находиться минимум. Чтобы теперь найти второй минимум посмотрим на путь соединяющий листовой камешек с корнем (длина такого пути будет составлять $\lceil \log_2 n \rceil$ рёбер) и выберем минимальный из камешков соседних с путем (то есть по сути, из всех камешков, которые из-за минимума отсеялись). Несложно убедиться, что таким образом мы найдем минимум среди оставшихся кроме нашего, то есть, по сути, второй минимум.

Чтобы выбрать минимум из $\lceil \log_2 n \rceil$ никак не связанных ранее друг с другом нам достаточно $(\lceil \log_2 n \rceil - 1)$ взвешиваний. Значит всего нам явно достаточно $(n + \lceil \log_2 n \rceil - 2)$ взвешиваний.

Таким образом, на данный момент имеем $2^{h(n)} = \mathcal{O}(n \cdot 2^n)$.

Теперь попробуем ограничить такой же оценкой снизу, тогда получим точную асимптотическую оценку. Для начала заметим, что как минимум $(n - 1)$ взвешивание точно понадобится, так как будем воспринимать камешки как максимально возможные по размеру группы камешков, где любой камешек сравнивался с хотя бы одним из группы. В начальный момент имеем n групп (каждый камешек в отдельной группе, так как пока что никто ни с кем не сравнивался). Когда мы делаем одно взвешивание мы уменьшаем количество групп не более чем на одну (ровно на одну, если камешки из двух разных групп, или на ноль если из одной). В конце нам обязательно иметь ровно одну группу, значит как минимум $(n - 1)$ взвешивание нам понадобится.

Рассуждение о том, что второй минимум является минимальным камешком среди тех, кто сравнивался с первым минимумом, остается верным, так как всегда есть взвешивание после которого минимум и второй минимум стали находиться в одной компоненте, тогда они были обязаны сравниться, так как являются минимумами в своих компонентах.

Ограничение снизу, в общем, я пока что не придумал.

□